GUARDIÕES DA LUZ

Documentação Técnica — Sprint 4
Domain Driven Design Using Java

Equipe NUVA
Matheus Costa Rodrigues dos Santos — RM 573146
Turma 1TDSPV

FIAP — Challenge 2026 — 2º Semestre

1. Objetivo e Escopo do Projeto

O **Guardiões da Luz** é a solução desenvolvida pela equipe NUVA para o Challenge FIAP 2026, com foco em engajar comunidades em ações de sustentabilidade (energia, água, floresta e resíduos) por meio de um sistema de missões gamificado. Usuários se cadastram em uma comunidade, completam missões relacionadas a hábitos sustentáveis e acumulam **SoulCoins**, uma moeda virtual conversível em benefícios reais (100 SoulCoins equivalem a R$ 1,00).

Esta entrega corresponde à **Sprint 4 da disciplina Domain Driven Design Using Java**: a evolução do projeto Java da Sprint 3 para uma **API RESTful completa, construída com Spring Boot**, pronta para ser consumida pelo front-end React da disciplina de Front-End Design Engineering. O escopo desta entrega cobre: persistência completa das 5 entidades do domínio (Comunidade, Missao, Usuario, UsuarioMissao e Pontuacao), as regras de negócio herdadas e adaptadas da Sprint 3, e os endpoints REST necessários para o funcionamento integral da aplicação.

O banco de dados utilizado é o schema Oracle entregue na Sprint 4 de Building Relational Database (tabelas `COMUNIDADE`, `MISSAO`, `USUARIO`, `USUARIO_MISSAO` e `PONTUACAO`), garantindo coerência entre as entregas de Banco de Dados e de Java.

2. Principais Funcionalidades

- **Gestão de Comunidades** — cadastro, consulta, atualização, remoção e cálculo da(s) comunidade(s) vencedora(s) por maior pontuação acumulada.
- **Gestão de Missões** — CRUD completo de missões, classificadas por categoria (Energia, Água, Floresta, Resíduos) e tipo (Onboarding, Diária, Círculo, Campo).
- **Gestão de Usuários** — cadastro vinculado a uma comunidade, com saldo de SoulCoins calculado dinamicamente a partir dos lançamentos ativos de pontuação.
- **Execução de Missões** — registro da conclusão de uma missão por um usuário, aplicando as regras de negócio: missão de Onboarding só pode ser concluída uma única vez; missão Diária não pode repetir no mesmo dia; missão de Campo exige validação manual antes de liberar a recompensa.
- **Validação de missões de Campo** — endpoint dedicado para aprovar ou rejeitar execuções pendentes (ex.: validação por vídeo).
- **Geração de SoulCoins** — ao validar uma execução de missão, o sistema calcula a recompensa (incluindo bônus de +50 SoulCoins a cada 7 dias consecutivos de missões diárias), gera o lançamento em PONTUACAO e atualiza a pontuação total da comunidade.
- **Bônus social e ajustes manuais** — lançamentos extras de SoulCoins por engajamento social (blocos de 10 reações = +5 SoulCoins) ou por ajuste administrativo.
- **Extrato e saldo do usuário** — consulta do histórico completo de pontuações e do saldo atual, já convertido em crédito em reais.
- **Tratamento global de exceções** — todas as violações de regra de negócio e buscas sem resultado retornam respostas HTTP padronizadas (400 e 404), nunca um erro não tratado.
- **CORS liberado** — a API aceita requisições do front-end React independentemente da origem, via filtro dedicado.

3. Tabela de Endpoints (API RESTful)

Todos os endpoints estão documentados interativamente via Swagger/OpenAPI em `/swagger-ui/index.html`. A tabela abaixo resume recurso, verbo, URI e os códigos de status HTTP esperados.

Recurso	Verbo	URI	Status esperados
Comunidade	GET	`/api/comunidades`	200
Comunidade	GET	`/api/comunidades/{id}`	200, 404
Comunidade	GET	`/api/comunidades/vencedoras`	200
Comunidade	POST	`/api/comunidades`	201, 400
Comunidade	PUT	`/api/comunidades/{id}`	200, 400, 404
Comunidade	DELETE	`/api/comunidades/{id}`	204, 404
Missao	GET	`/api/missoes`	200
Missao	GET	`/api/missoes/{id}`	200, 404
Missao	POST	`/api/missoes`	201, 400
Missao	PUT	`/api/missoes/{id}`	200, 400, 404
Missao	DELETE	`/api/missoes/{id}`	204, 404
Usuario	GET	`/api/usuarios`	200
Usuario	GET	`/api/usuarios/{id}`	200, 404
Usuario	POST	`/api/usuarios`	201, 400
Usuario	PUT	`/api/usuarios/{id}`	200, 400, 404
Usuario	DELETE	`/api/usuarios/{id}`	204, 404
Execucao	GET	`/api/execucoes`	200
Execucao	GET	`/api/execucoes/{id}`	200, 404
Execucao	GET	`/api/execucoes/usuario/{usuarioId}`	200
Execucao	POST	`/api/execucoes`	201, 400, 404
Execucao	PUT	`/api/execucoes/{id}/avaliar`	200, 400, 404
Execucao	DELETE	`/api/execucoes/{id}`	204, 404
Pontuacao	GET	`/api/pontuacoes/usuario/{usuarioId}`	200
Pontuacao	GET	`/api/pontuacoes/usuario/{usuarioId}/saldo`	200
Pontuacao	POST	`/api/pontuacoes/bonus-social`	201, 400, 404
Pontuacao	POST	`/api/pontuacoes/ajuste`	201, 400, 404
Pontuacao	DELETE	`/api/pontuacoes/{id}`	204, 404

4. Protótipo — Evidências de Execução

O front-end React (disciplina de Front-End Design Engineering) ainda não está integrado no momento desta entrega. Como evidência funcional da API, seguem capturas reais da aplicação rodando localmente e consumindo o banco Oracle da FIAP.

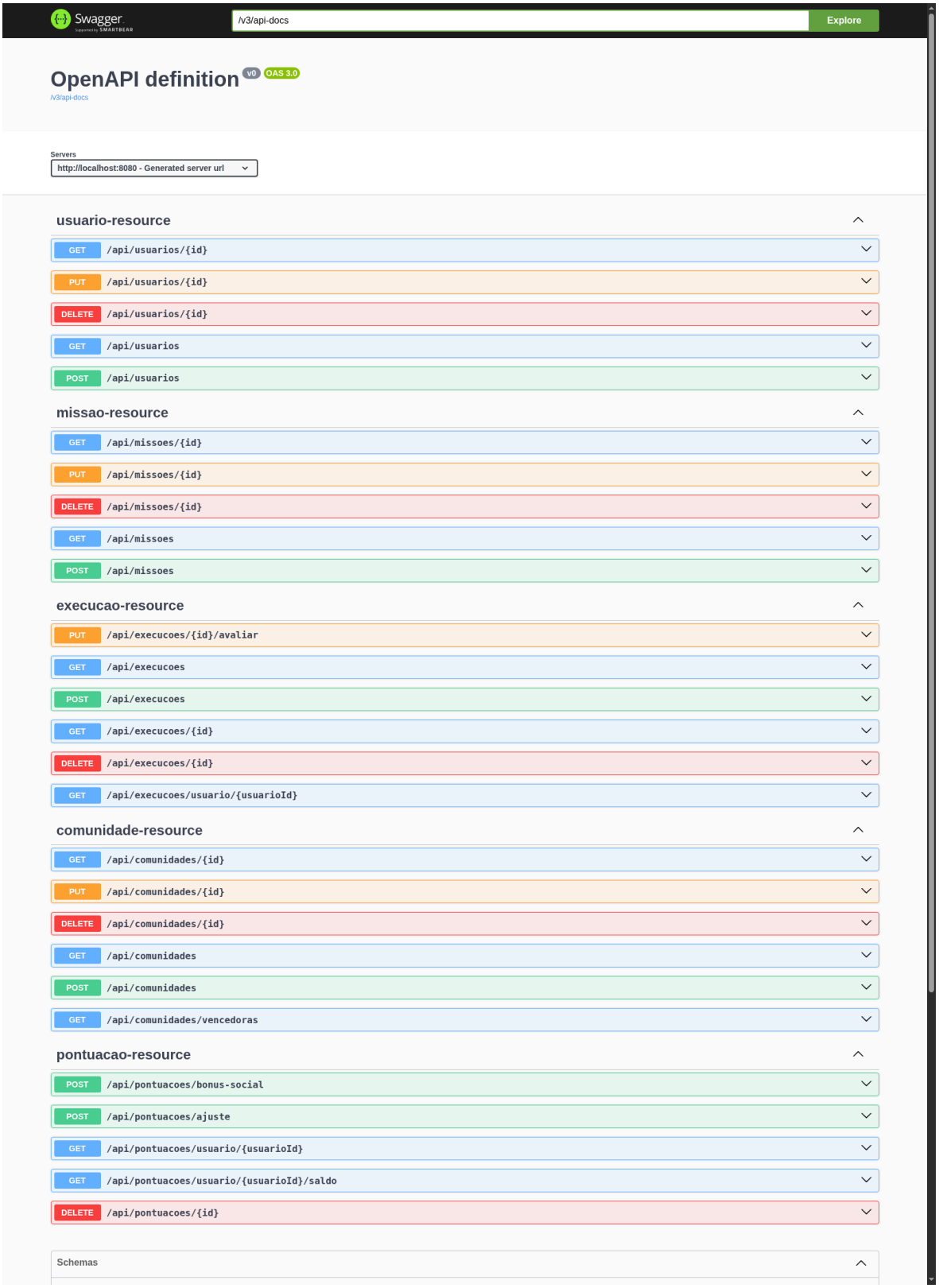


Figura 1 — Documentação interativa (Swagger UI) com os 27 endpoints da API, agrupados pelos 5 controllers (Resource).

Abaixo, uma sessão real de terminal exercitando a API: uma consulta bem-sucedida (200), uma violação de regra de negócio capturada pelo handler global de exceções (400 — tentativa de repetir uma missão de onboarding já concluída) e uma busca por um recurso inexistente (404).

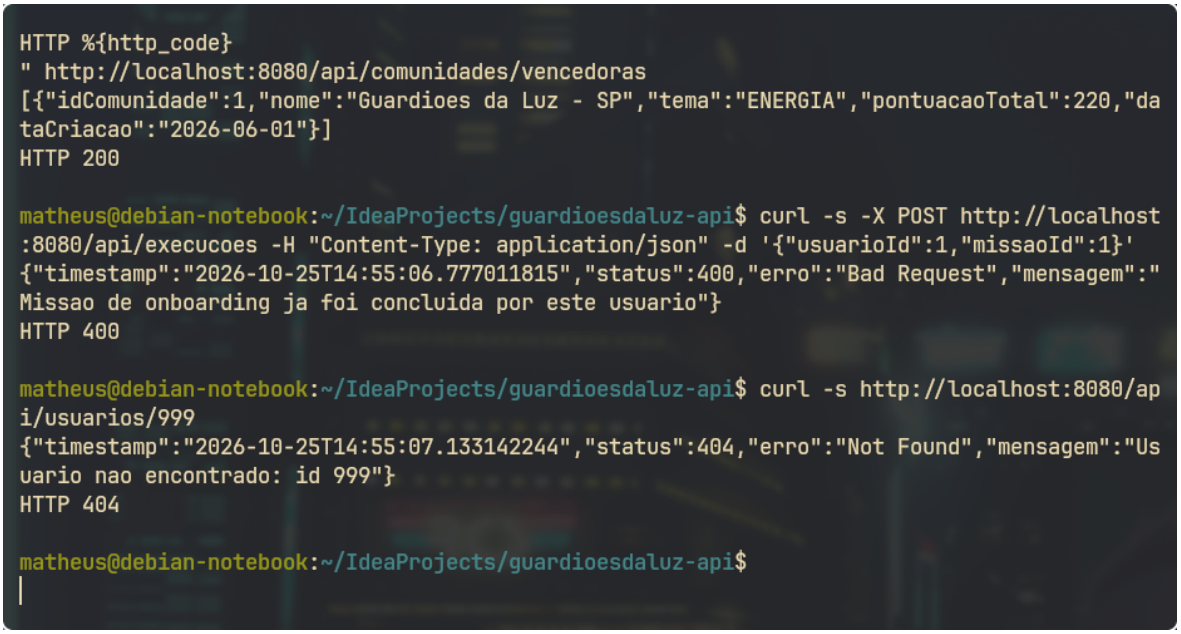


Figura 2 — Execução real via curl contra a API rodando em localhost:8080, com respostas HTTP 200, 400 e 404.

5. Modelo de Entidade-Relacionamento (MER)

O diagrama abaixo reflete o schema físico entregue na Sprint 4 de Building Relational Database, base utilizada pela camada de persistência (DAO) desta aplicação.

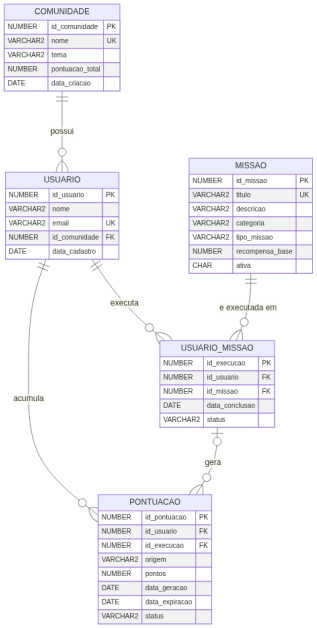


Figura 3 — MER com as 5 tabelas, chaves primárias/estrangeiras e relacionamentos (COMUNIDADE 1:N USUARIO, USUARIO 1:N USUARIO_MISSAO, MISSAO 1:N USUARIO_MISSAO, USUARIO 1:N PONTUACAO, USUARIO_MISSAO 0..1:N PONTUACAO).

6. Diagrama de Classes Atualizado

Representação UML da camada `model` do projeto Java: as 5 classes de entidade, seus atributos, principais métodos de negócio e os enums que espelham os CHECK constraints do banco de dados.

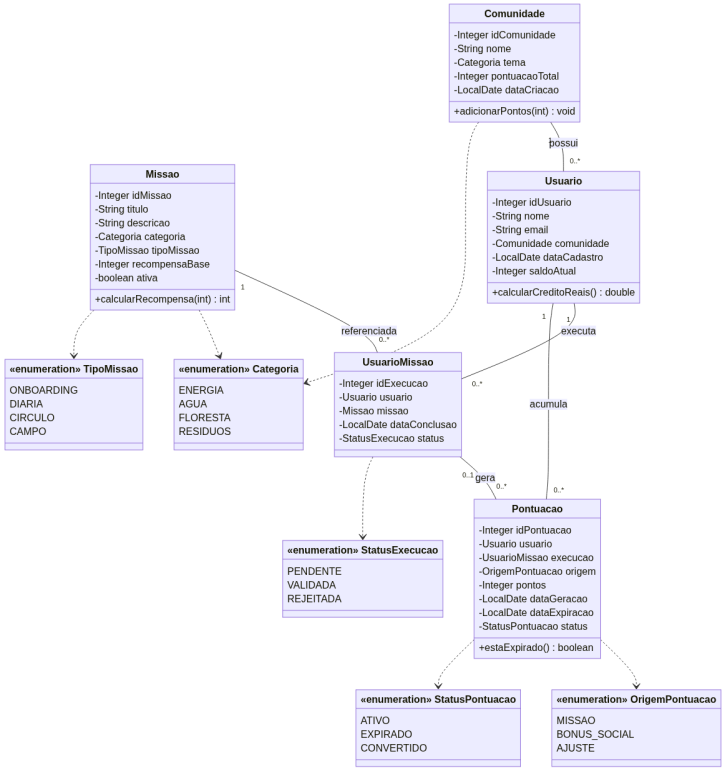


Figura 4 — Diagrama de classes das entidades do domínio (br.com.fiap.guardioesdaluz.model), compatível com o código-fonte entregue.

7. Procedimentos para Rodar a Aplicação

Ferramentas utilizadas

- IntelliJ IDEA (Community ou Ultimate)
- JDK 17 ou superior
- Apache Maven (o próprio IntelliJ já traz um Maven embutido, não é obrigatório instalar separadamente)
- Acesso à internet (para resolução das dependências Maven) e ao banco Oracle da FIAP (oracle.fiap.com.br:1521)

Como importar o projeto

- Abrir o IntelliJ IDEA → *File* → *Open* → selecionar a pasta raiz do projeto (onde fica o `pom.xml`).
- O IntelliJ reconhece automaticamente o projeto como Maven e baixa as dependências declaradas no `pom.xml` (Spring Boot Web, driver Oracle JDBC, Springdoc OpenAPI).

Passo a passo da execução

- 1. Aguardar o IntelliJ terminar a importação/indexação do Maven.
- 2. Localizar a classe `GuardioesDaLuzApplication` em `src/main/java/br/com/fiap/guardioesdaluz`.
- 3. Executar essa classe (botão Run ou Shift+F10) — ela sobe um servidor Spring Boot embutido (Tomcat) na porta 8080.
- 4. A aplicação conecta automaticamente ao Oracle da FIAP através da `ConnectionFactory` (credenciais já configuradas no código).
- 5. Testar os endpoints via Swagger em `http://localhost:8080/swagger-ui/index.html`, ou via Postman/curl apontando para `http://localhost:8080/api/...`

Caso necessário, como rodar via linha de comando

```
cd guardioesdaluz-api
mvn spring-boot:run
```

O projeto também pode ser empacotado como JAR executável com `mvn clean package` e rodado com `java -jar target/guardioesdaluz.jar`.